

Product of Array Except Self

<https://leetcode.com/problems/product-of-array-except-self/>

Family Packing for a Trip

The Situation:

A family of 4 — **Papa, Mummy, Sonu, and Monu** — is packing bags for a road trip. Each person is responsible for packing a **certain number of essential items**:

`bags = [2, 3, 5, 4]`

- Papa packed 2 items
- Mummy packed 3
- Sonu packed 5
- Monu packed 4

The Mission:

Mummy wants to know:

"If I remove **each** person's contribution, how much effort did **everyone else** make *together*?"

In other words: **product of all other contributions**, for **each** family member — but without using division!

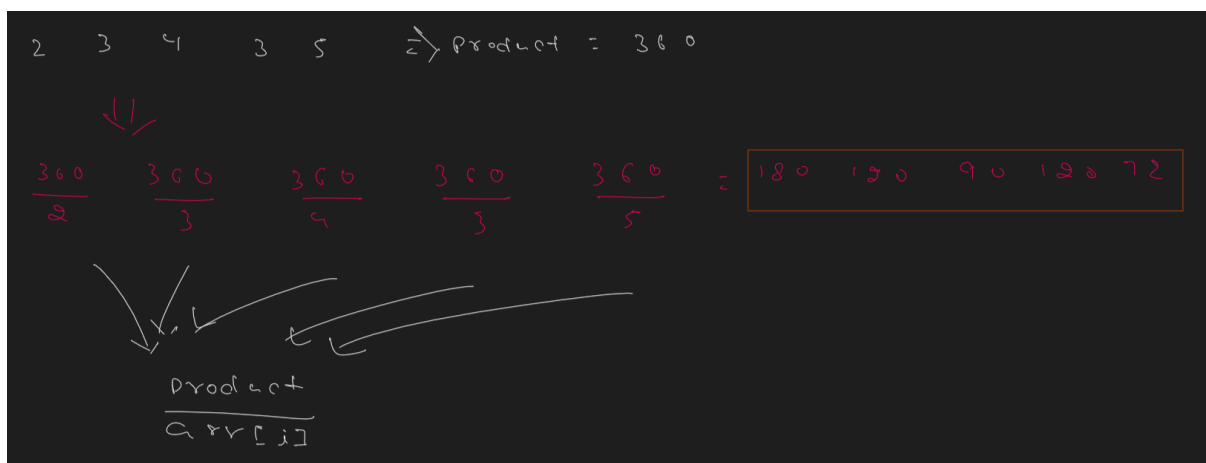
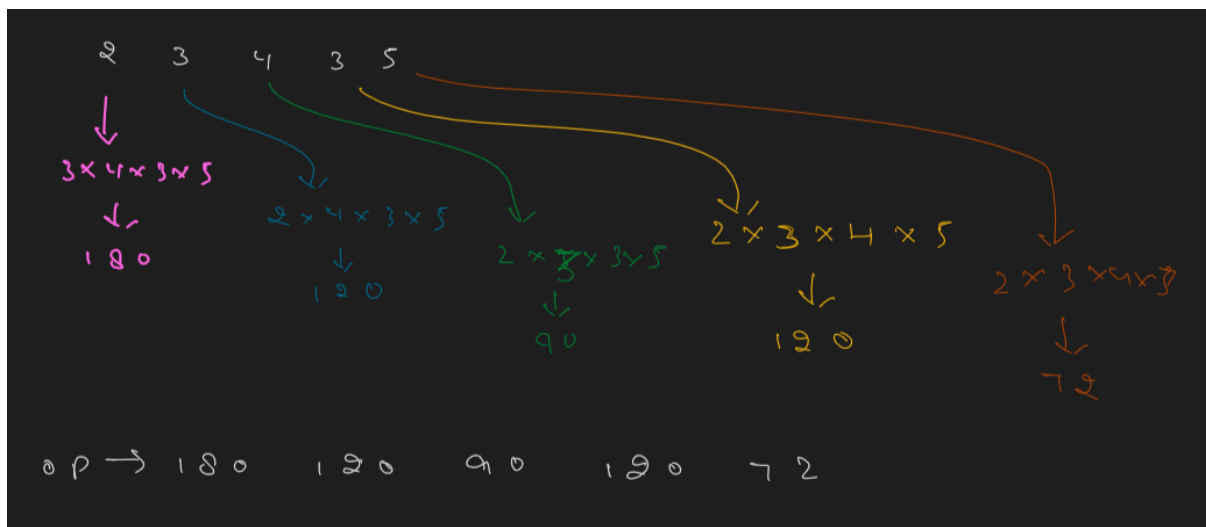
(Because Indian moms don't divide — they **delegate smartly** 😎)

Desired Output:

- Papa's contribution removed: $3 \times 5 \times 4 = 60$
- Mummy's removed: $2 \times 5 \times 4 = 40$
- Sonu's removed: $2 \times 3 \times 4 = 24$
- Monu's removed: $2 \times 3 \times 5 = 30$

Final result: `[60, 40, 24, 30]`

Brute Force Solution: *Using Division Operator*



```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 3, 4, 3, 5};
        // System.out.println(product(arr));

        int[] nums = productOfArrayExceptSelf(arr);

        for (int i = 0; i < nums.length; i++) {
            System.out.print(nums[i] + " ");
        }
    }

    public static int product(int[] arr) {
        int prod = 1;
        int n = arr.length;
        for (int i = 0; i < n; i++) {
            prod = prod * arr[i];
        }
        return prod;
    }

    public static int[] productOfArrayExceptSelf(int[] arr) {
        int prod = product(arr);
        int n = arr.length;
        int[] op = new int[n];

        for (int i = 0; i < n; i++) {
```

```

        op[i] = prod / arr[i];
    }
    return op;
}
}

```

🧠 Real-Life Jugaad: How Mummy Thinks

She doesn't divide. She uses "left and right contribution tracking":

✳️ Step 1: Left-side Contribution (before each person)

$left = [1, 2, 2 \times 3 = 6, 6 \times 5 = 30] \rightarrow [1, 2, 6, 30]$

✳️ Step 2: Right-side Contribution (after each person)

$right = [3 \times 5 \times 4 = 60, 5 \times 4 = 20, 4, 1] \rightarrow [60, 20, 4, 1]$

🌟 Final Output:

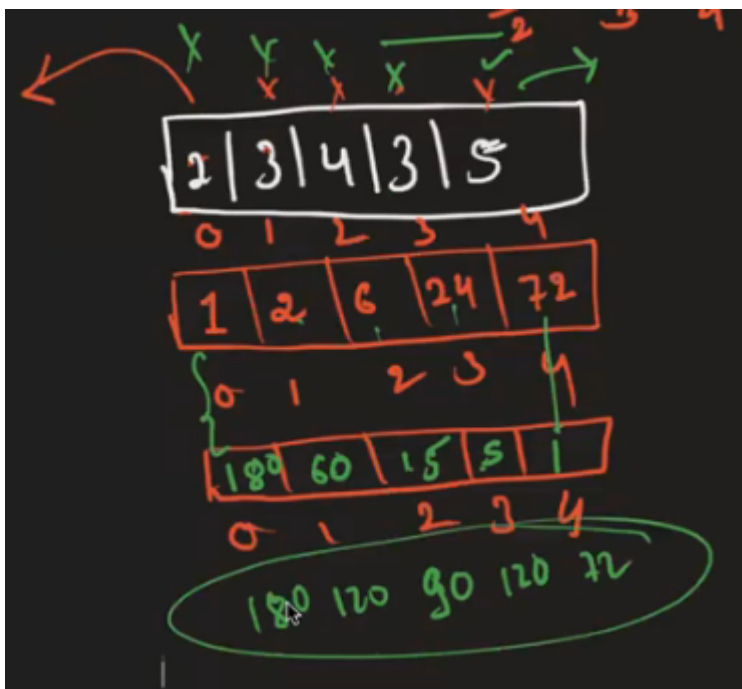
$result = [1 \times 60, 2 \times 20, 6 \times 4, 30 \times 1] \rightarrow [60, 40, 24, 30]$

😊 Mummy's Wisdom:

"Now I know who's slacking and who's actually pulling their weight! Next trip, I'm giving Papa fewer tasks!" 😊

✅ Why This Analogy Works:

- Each person's role matters, but we want to measure their absence's effect.
- In real life, we often want to know: "If this person didn't help, how much would be left for others?"



To solve this problem, I need to find the **prefix and suffix product arrays**. Suppose I'm calculating the left (prefix) array — **for each element, I will leave out that specific element and calculate the product of all the elements to its left**. I'll do the same for the right (suffix) array — leaving the current element and finding the product of all the elements to its right. The final result for each position will be the product of the corresponding values from the left and right arrays.

$$\begin{matrix} 1 & 2 & 6 & 24 & 120 \\ \hline \end{matrix}$$

$$\begin{matrix} 0 & 1 & 2 & 3 & 4 \\ \hline 1 & 18 & 60 & 15 & 5 & 1 \end{matrix}$$

$$\begin{aligned} \text{left}(1) &= \text{left}(0) \times \text{arr}(0) \\ \text{left}(2) &= \text{left}(1) \times \text{arr}(1) \\ \text{left}(3) &= \text{left}(2) \times \text{arr}(2) \\ \text{left}(4) &= \text{left}(3) \times \text{arr}(3) \end{aligned}$$

$$\begin{aligned} \text{right}(3) &= \text{right}(4) \times \text{arr}(4) \\ \text{right}(2) &= \text{right}(3) \times \text{arr}(3) \\ \text{right}(1) &= \text{right}(2) \times \text{arr}(2) \\ \text{right}(0) &= \text{right}(1) \times \text{arr}(1) \end{aligned}$$

$$\text{right}(0) = \text{right}(1) \times \text{arr}(1)$$

$$\text{left}(i) = \text{left}(i-1) \times \text{arr}(i-1)$$

```

public class Main {
    public static void main(String[] args) {
        int[] arr = { 1, 2, 3, 4 };
        int[] a = productOfArray(arr);
        for (int i = 0; i < a.length; i++) {
            System.out.print(a[i] + " ");
        }
    }

    public static int[] productOfArray(int[] arr) {
        int n = arr.length;
        int[] left = new int[n];
        int[] right = new int[n];

        left[0] = 1;
        for (int i = 1; i < n; i++) {
            left[i] = left[i - 1] * arr[i - 1];
        }

        right[n - 1] = 1;
        for (int i = n - 2; i >= 0; i--) {
            right[i] = right[i + 1] * arr[i + 1];
        }

        for (int i = 0; i < n; i++) {
            left[i] = left[i] * right[i];
        }

        return left;
    }
}

```

```

class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
    }
}

```

```
// Arrays to store left and right products
int[] left = new int[n];
int[] right = new int[n];

// left[i] contains product of all elements to the left of index i
left[0] = 1;
for (int i = 1; i < n; i++) {
    left[i] = left[i - 1] * nums[i - 1];
}

// right[i] contains product of all elements to the right of index i
right[n - 1] = 1;
for (int i = n - 2; i >= 0; i--) {
    right[i] = right[i + 1] * nums[i + 1];
}

// Multiply left and right products for the final result
for (int i = 0; i < n; i++) {
    left[i] = left[i] * right[i];
}

return left; // left now contains the result
}
```

```
}
```